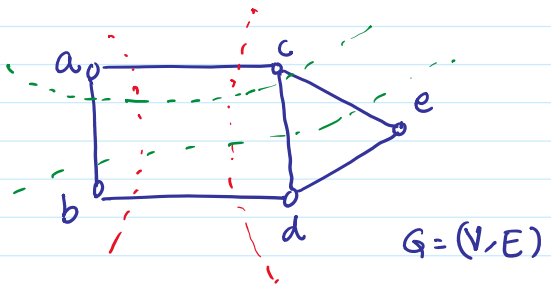
11) | Lecture 21 |MST and its solution using Prim's Algorithm |

18 September 2026 11:33

Cut in a graph (Undirected)

A cut in G is partition of vertex set V in to parts – S, V/S



	S	V/S
Possible cut	{a, b}	{c, d, e}
	{a, c}	{b, d, e}

S, V/S are non empty.

of possible cuts $2^n - 2$

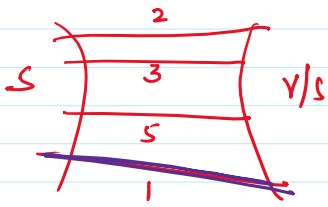
$\bar{S} := V/S$

Assumption: All edges are of distinct weight.

Consider weighted graph $G = (V, E)$ $Wt(e) \rightarrow \mathbb{R}^+$ $\forall e \in E$

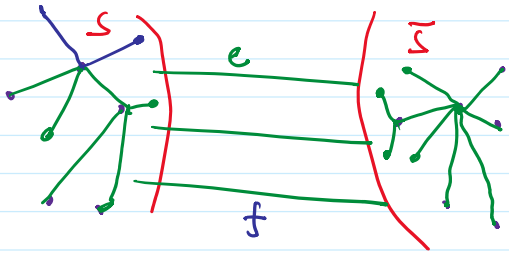
Claim For any cut $(S, \bar{S})$, the min weight edge across partition must belong to MST.

More than one edge can also be part of MST but min weight edge must be part of MST.



Proof

Consider a cut $(S, \bar{S})$. Let e be min wt edge across the partition.



Assume contrary that ∃ a MST T that doesn't edge e

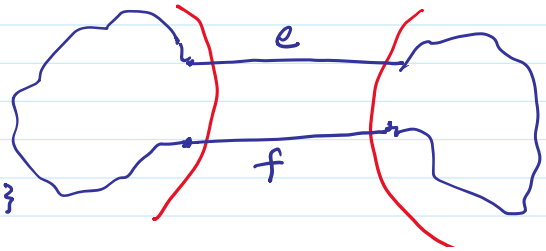
MST T has some edge f across the partition s.t $Wt(f) > Wt(e)$ G (By assumption)

Consider $T' \leftarrow T \cup \{e\}$

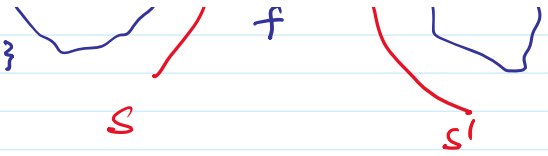
T' must have a cycle.

remove edge f from T'

$T'' \leftarrow T' \setminus \{f\}$ $T'' \leftarrow T \cup \{e\} \setminus \{f\}$

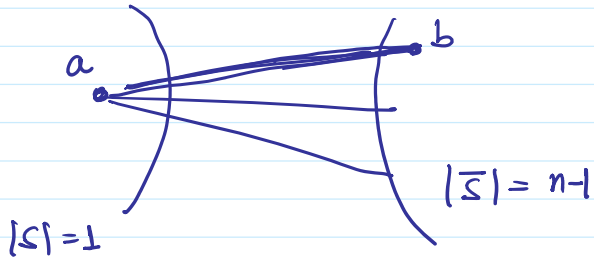


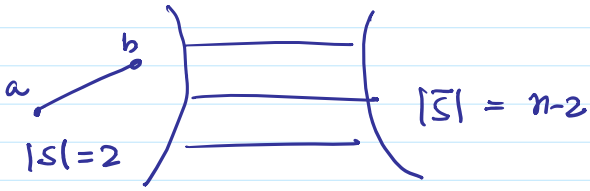
$T'' \leftarrow T' \setminus \{f\}$ $\quad$ $T'' \leftarrow T \cup \{e\} \setminus \{f\}$



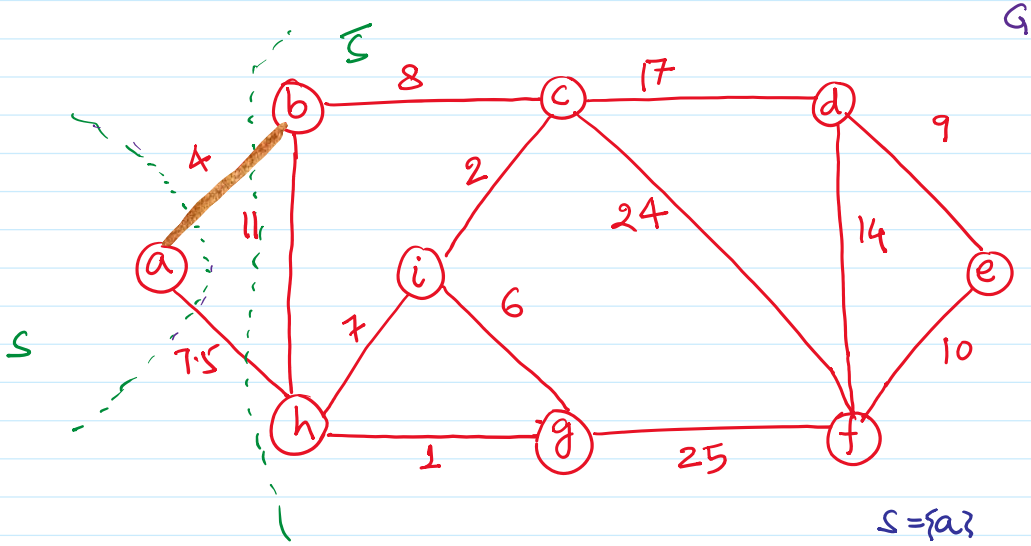
$\Rightarrow \quad wt(T'') < wt(T)$ $\quad$ contradiction

At every time step set S consist of MST of vertices $\in S$



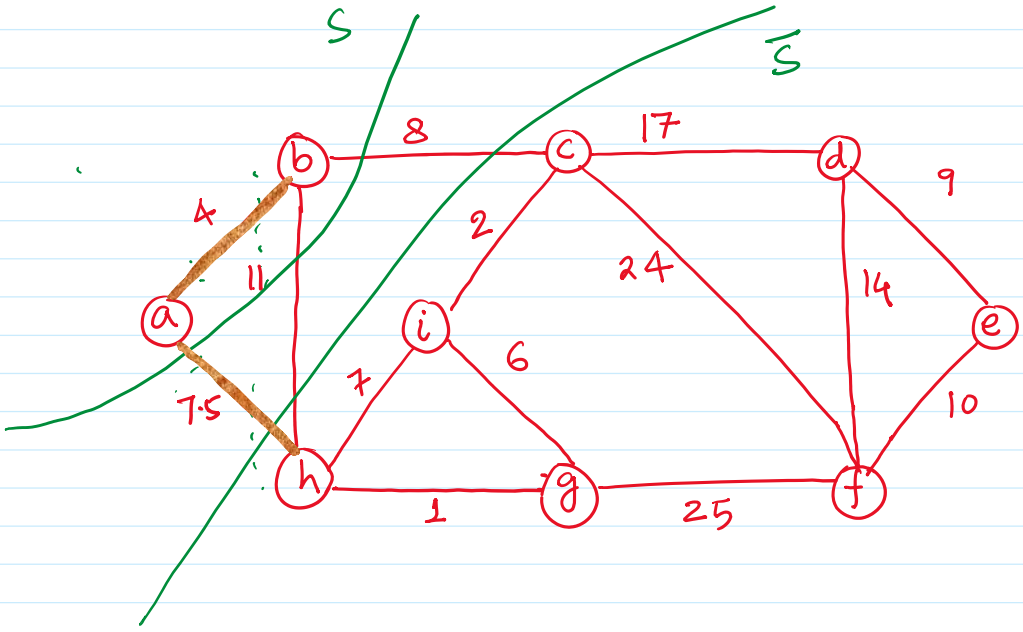


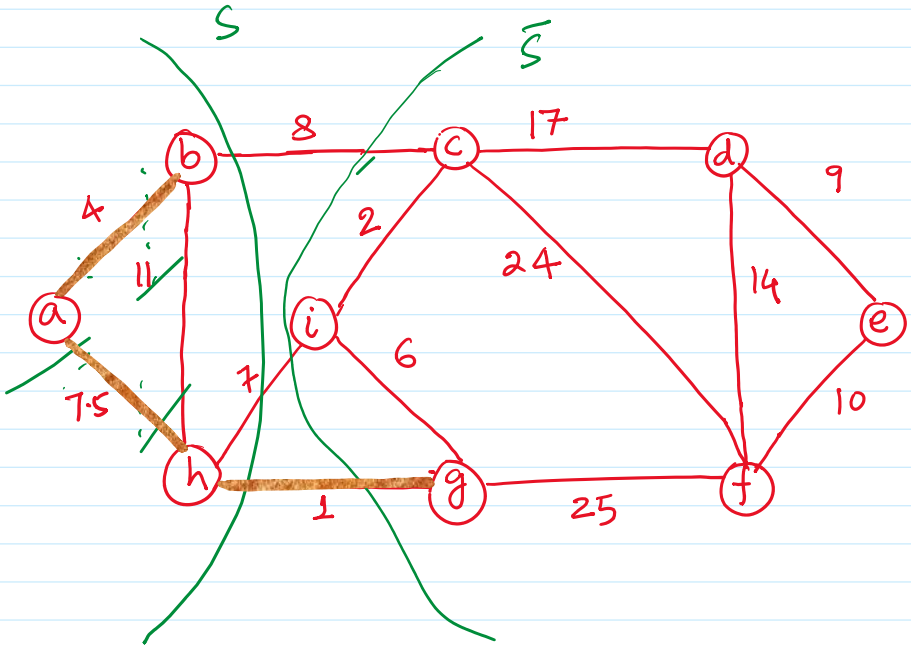
$G = (V, E)$

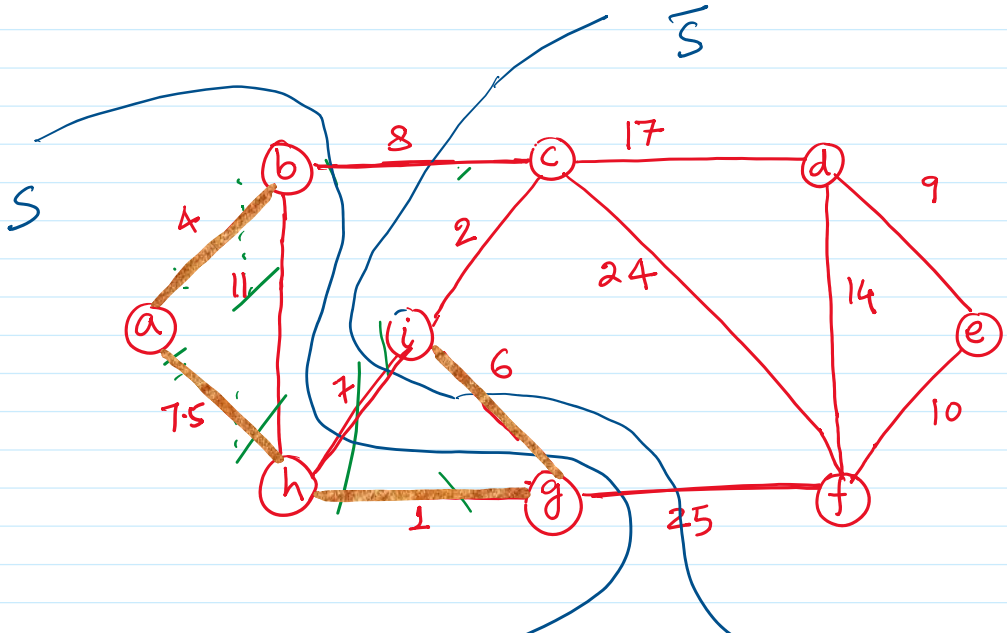


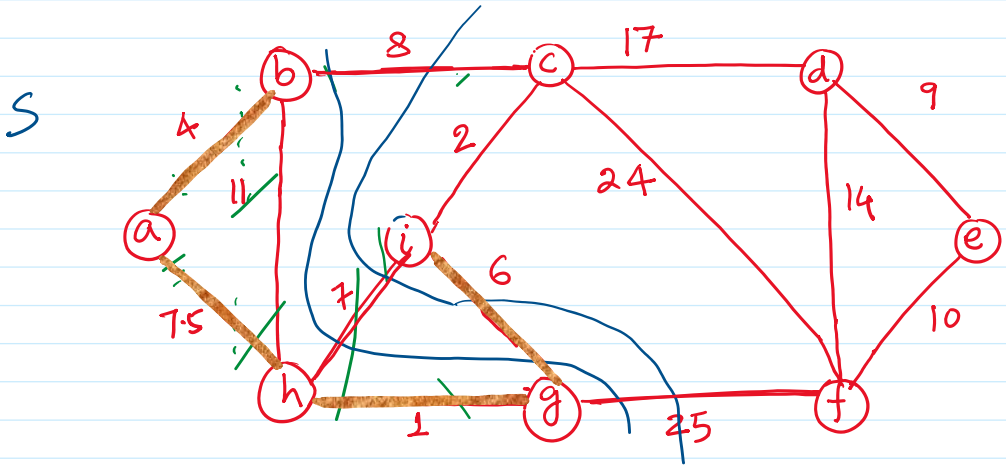
$S = \{a\}$ $\quad$ $\bar{S} = V \setminus S$

S	$\bar{S}$
a	$V \setminus a$

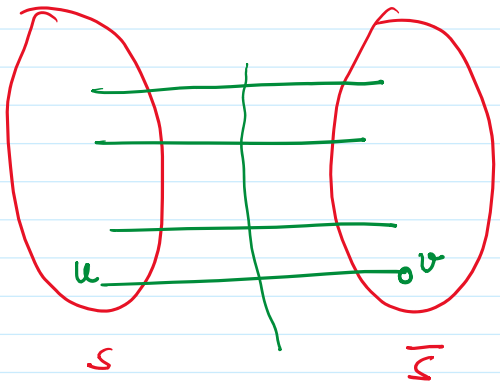

S
S̄
a
b
c
d
e
f
g
h
i
4
8
17
9
11
2
24
14
7
6
7.5
10
1
25


S
S̄
a
b
c
d
e
f
g
h
i
4
8
17
9
11
2
24
14
7
6
7.5
10
1
25

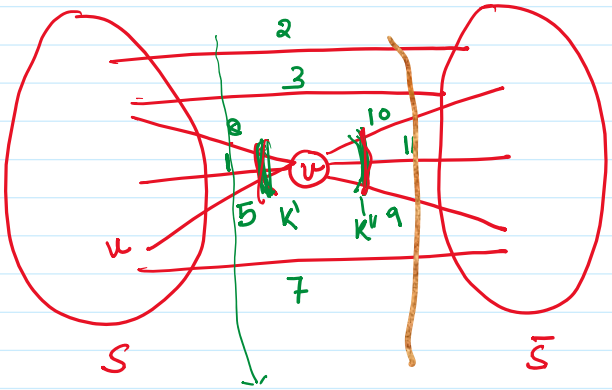

S
S̄
a
b
c
d
e
f
g
h
i
4
8
17
9
11
2
24
14
7
6
7.5
10
1
25



What we do in every step – look at edges across the partition and pick the edge of min weight and include in MST.



let uv is min weight edge.



know min of all these edges

need to compute min of these edges

represented by X

You have clever alg that takes K no. and o/p their min

in each iteration compute $K - K' + K''$ no. (min of)

Heap is a data structure that output min of m no in O(1) time

inclusion and exclusion of a new element in set require $O(\log m)$ time

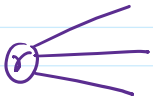
S is an array

$S[v] = 1$ if $v \in S$

0 o.w.

$\forall v \in V$ $S[v] = 0$ initialization

$S[r] = 1$ /* root vertex */

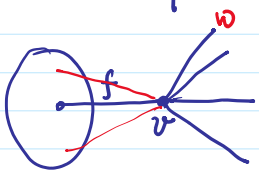


$\forall e$ incident to r do

H.insert(e) /* H.insert(e) is fn to include e in heap

While !H.empty() do

$f \leftarrow$ H.findMin() /* H.findMin() o/p min of heap



Let v be the end pt of f s.t.

$S[v] = 0$

$\forall$ all edge $e = (v, w)$ adjacent to v

if $(S(w) == 0)$

H.insert(e)

else

H.delete(e) /* $v \in \bar{S}$ */

$S[v] \leftarrow 1$

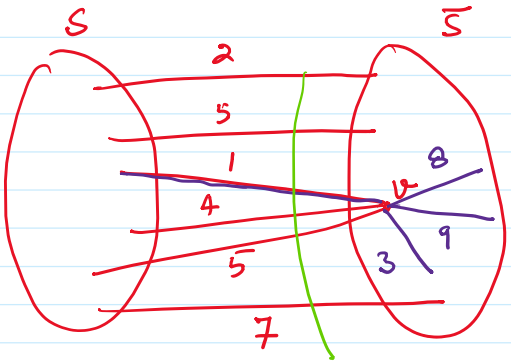
creating heap over edges across partition

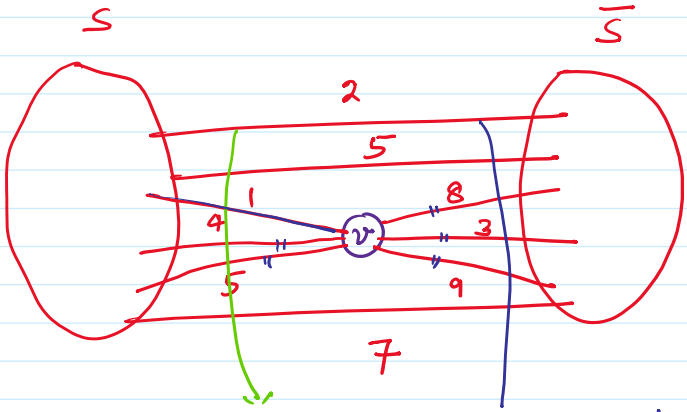
$m = O(n^2)$

Overall time complexity = $\underbrace{n\, O(1)}_{\text{finding Min}} + \sum_{v \in V} \deg(v) \log m$

$= O(n + m \log m)$







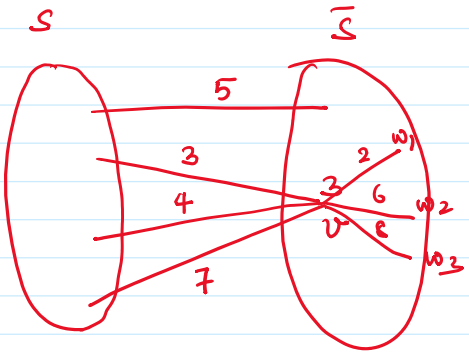
we know min of these edges

we want to compute min of these edges

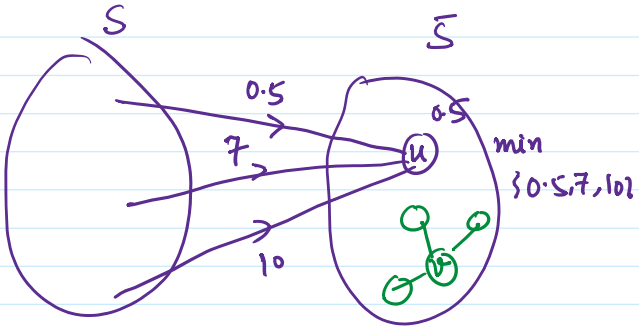
Modification Each vertex in $\bar{S}$ has a label which is wt of min edge across partition incident to it.

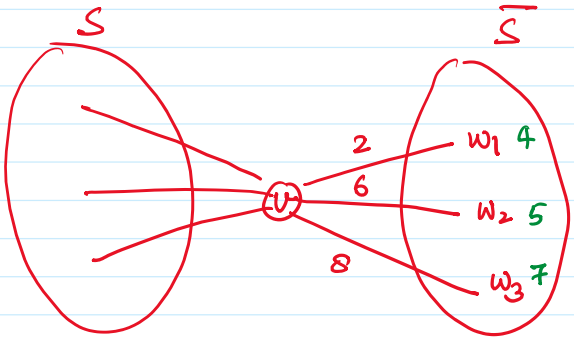
We will create heap of these label vertex

When we move a vertex from $\bar{S}$ to S we update label of vertices in $\bar{S}$.

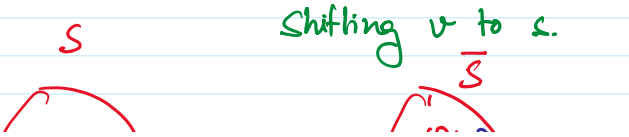


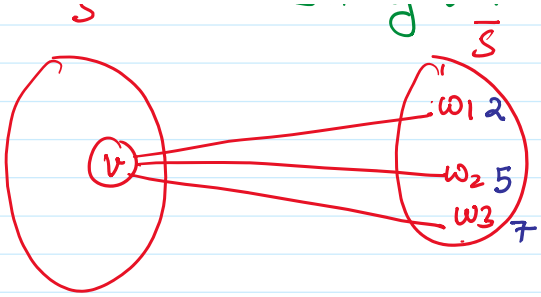
Heap will contain vertices of $\bar{S}$ only.





4,5,7 are original label of w_i before shifting v to S.





S is an array

S[v] = 1 if $v \in S$

0 o.w.

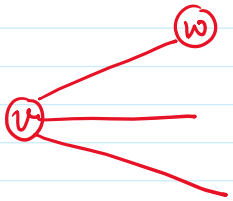
$\forall v \in V$ S[v] = 0 , H.insert(v, ∞)

S[r] = 1 /* root vertex */ H.decreasepriority(r, 0)

vertex, updated label

While !H.empty() do

v = H.findMin()



for all w adjacent to v do{

if (S[w] == 0), then /* w belong to $\bar{S}$

if label(w) > wt(v,w) wt(v,w) := weight of edge (v,w)

then label[w] = wt(v,w)

H.decreasepriority[w, label(w)]

else

H.delete[w, label(w)]

S[v] ← 1

}